AGENTIC AI FOR INFORMATION RETRIEVAL: A COMPARATIVE STUDY



ANINDYA SUNDER CHAKRAVARTY (D03000004)

INTRODUCTION

- Naïve RAG systems execute a fixed retrieval pipeline. Due to this limitation, it often fails at fulfilling complex information requirements.
- Agentic AI introduces an autonomous design which enables a model to observe the information available (such as the query) and formulate and execute a plan through the usage of tools.
- It decides to use tools based on the situation to complete the retrieval objectives.
- This project is a comparative analysis of two Agentic Retrieval systems
  - one operating on a static corpus
  - one extracting information real time from an online source.
- Both approaches have their own methodologies, advantages and faults.

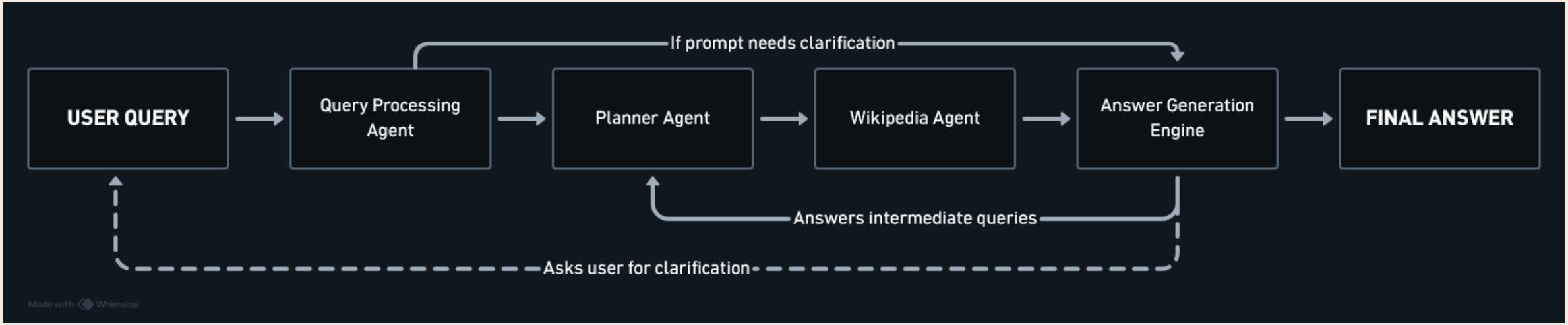
SYSTEM IMPLEMENTATION

Core components of the system:

- Query Handler – Responsible for interpreting, decomposing and extracting information from the user's query. If necessary, it can also make a request for clarification.
- Planner – It receives the Query Handler's decomposed query with added information as input and devises a plan to fulfil the user's information need. It splits query into multiple objectives and generates seed queries for each objective. The planner is also capable of orchestrating complicated questions' retrieval.
- Retrieval Provisions – As discussed before we have two different approaches for each pipelines.
  - Static Corpus Pipeline – Hybrid Retrieval System combines both dense and sparse retrieval and applies neural ranking to maximise evidence precision
  - Real Time Information – Dedicated agent built around the Wikipedia REST API that can search, extract, evaluate and present information for individual seed queries
- Answer Generation Engine – Responsible for interfacing with the user. Primary response is to forming an answer to the user's query based on presented evidence. Secondary objectives include interacting with the planning agent to supply intermediate objective queries' answers and based on the query planner's output request the user for further clarification on their query.
- Evaluation System – Detached evaluation system inspired from RAGAS evaluation framework, based on the 2WikiMultiHopQA dataset.
- LLM Service Provider Interface – Enables using both Local LLM setups via LM Studio and cloud LLMs via Open Router API.

SYSTEM IMPLEMENTATION

Core components of the system:



SYSTEM IMPLEMENTATION

QUERY HANDLER

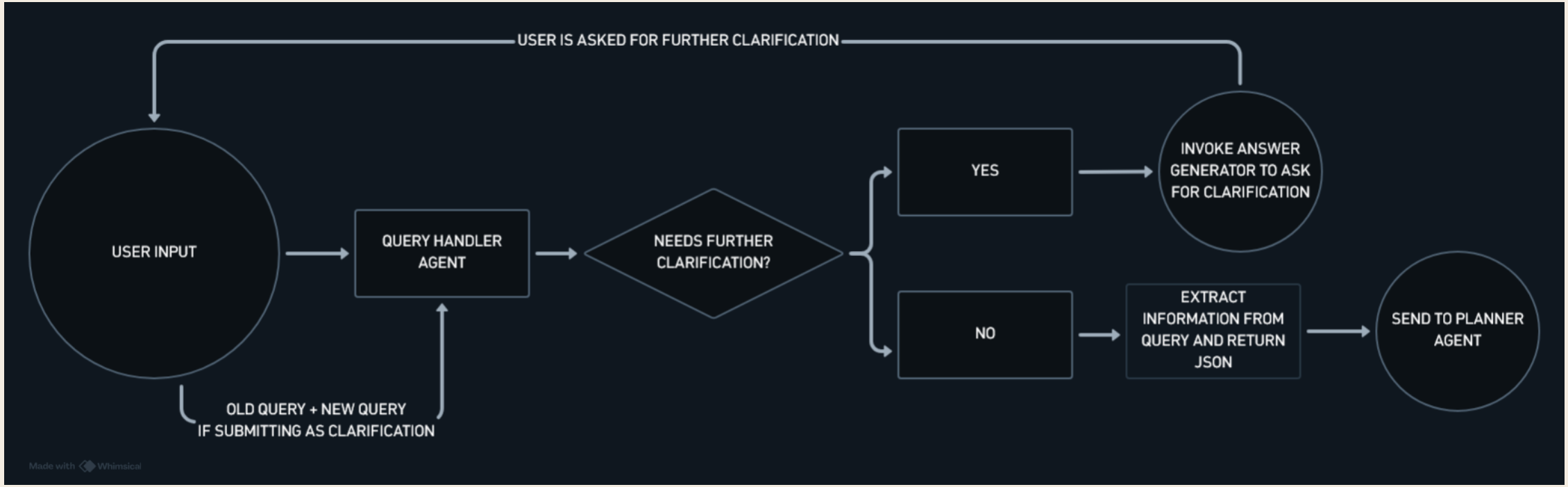
- Primary Responsibility: Understand user's query, extract information from it and return a structured response packaging all the information it has extracted about the query.
- Secondary Responsibility: After processing the user's query, the agent may choose to request the user to provide clarification on their query.
- Statefulness: If clarification is triggered, for every provided clarification, both the old and the 'new' query is supplied to the agent as conversation history. For example:
  - Original Query: "*Who was he fighting?*" -> LLM: Please Clarify -> User: "*Napoleon*."
  - Without maintaining conversation history, the clarification itself ('*Napoleon*') has no meaning.
- Clarification Loop: The system is strictly controlled with hard termination limits. In this case the query handler can ask for a maximum of 3 clarifications. If the limit is exceeded, then the program is terminated.
- Gathered Information: The module is built to extract different types of information from the user's query:
  - *Intent* – Describes the information needed
  - *Entities* – Identifies people, organizations, locations etc.
  - *Concepts* – Identifies topics, phenomena etc.
  - *Relationships* – Applies subject-object-relationship mapping between entities

SYSTEM IMPLEMENTATION

QUERY HANDLER (continued)

- *Query*: Retains the user's query as it is
- *Status*: Signals the system if clarification is required or if information extraction was successful.

- Output: Returns a structured output packaging the above-mentioned information which is used downstream.
- Let us visualize the module below:



SYSTEM IMPLEMENTATION

PLANNER

- Responsibilities: Process the information supplied by the query handler module, build a retrieval strategy that consists of what objectives need to be fulfilled and seed queries for each objective. The module produces a retrieval plan containing a list of objectives, where each objective contains its seed queries and any dependencies.
- Sample Output: { "objectives": [{ "objective": "Who is the current president of Italy.", "depends_on": [], "seed_queries": ["Current President of Italy"] }, { "objective": "Where was the person referred to in Objective 1 born?", "depends_on": [1], "seed_queries": ["{previous_result} birthplace"] }] }
- Each objective contains a "depends on" field which may indicate if it depends on the intermediate answer for any other objective. For example, consider the question "Where was the current president of Italy born"? It needs two pieces of information one after the other ("*who is the current president of Italy?*" and "*where was* __answer from first question__ *born*"?) to be able to fully answer the question.

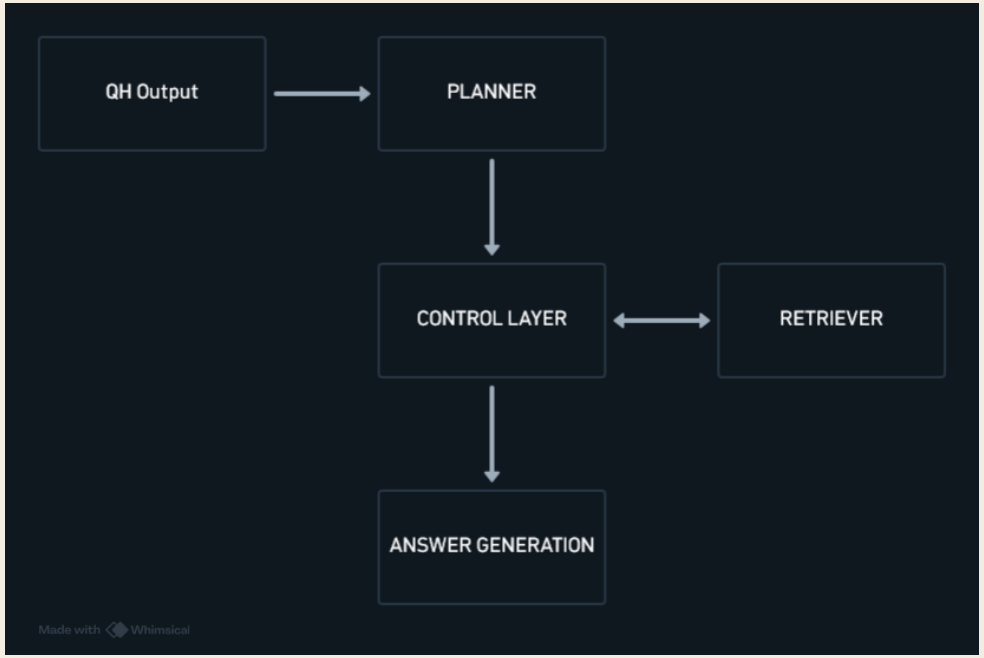
We can notice that through formatting its output like this it is able to link intermediate objectives together. The control layer uses this to fetch intermediate answers fill in the subsequent seed queries and lead towards the final answer.

- Seed Queries: As mentioned above, each objective contains one or more seed queries. These seed queries are used to query the retrieval systems instead of supplying entire user queries. Seed queries for dependent objectives are first generated with a "blank" and is replaced by the intermediate answer is obtained. Example: "{*previous_result*} birthplace"
- In the following slides we will look at how the agent works.

SYSTEM IMPLEMENTATION

PLANNER: HOW DOES IT WORK?

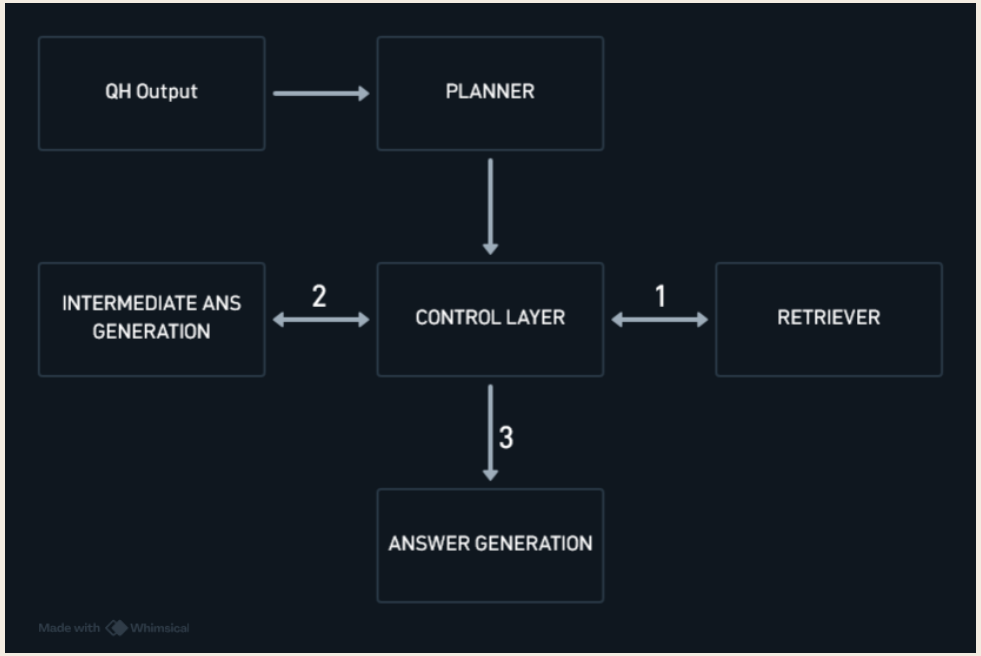
- Despite its functionality, the planner agent itself does not work in a recursive way. It generates the retrieval strategy once and exits. The rest of the functionality is handled by deterministic python code.
- Simple Queries: When the queries are simple, i.e., can be fulfilled with one objective and one set of seed queries, the control layer extracts the seed queries from the planner's output and invokes the retrieval system for each seed query. Once retriever returns the evidence, it is passed along to the answer generation module. Since the control loop only finds one element in the 'objectives' list it terminates here.



SYSTEM IMPLEMENTATION

PLANNER: HOW DOES IT WORK?

- **Complicated Queries:** When the query is complicated and has multiple objectives, then the control layer extracts the first set of seed queries, feeds them to the retriever one by one and gathers the evidence supplied. This evidence for the intermediate objective is then sent to the answer generation by calling a special function, which instructs the LLM to only return the minimum piece of information required based on the evidence supplied. Once this minimal answer is fetched it is replaced in the secondary objectives' seed queries' special field – e.g.: '{previous_result} birthplace' -> *'Sergio Mattarella* birthplace' and this new seed query is sent for retrieval and once all the objectives are complete, we send the evidence for answer generation.



SYSTEM IMPLEMENTATION

WIKIPEDIA AGENT

- Primary Responsibility: Fetch relevant information from Wikipedia based on the supplied seed query.
- Architecture: There are multiple subagents involved in each runtime, each tasked with a different objective within the information retrieval process.
  - Page Selection Subagent: Selects the pages based on Wikipedia search results which are most likely to contain relevant information.
  - Evidence Evaluation Subagent: Evaluates whether the gathered evidence is relevant to the search query. Can decide between marking evidence as sufficient, invoke the local query rewrite subagent or follow links extracted from page(s).
  - Query Rewrite Subagent: If invoked by the evidence evaluator, this subagent will rewrite the seed query to possibly get better results.
- Controlled Exploration: For each step there are hard termination limits in place which control the behaviour of the LLM such as maximum search attempts, maximum link following iterations, maximum links to follow per iteration. Moreover, there is a "visited links" filter that filters out the already visited pages before presenting them to the LLM for evaluation.
- Logging and Data Persistence: The pipeline's execution trace is continuously logged, and the final set of evidence is saved as well.
- State Handling: There are two kinds of state being maintained by the agent – "local state" handled by the control layer (visited pages, evidence from previous retrievals etc.) The second kind is conversational history which is only used for query rewriting.

SYSTEM IMPLEMENTATION

WIKIPEDIA AGENT

- **Focus on controlling API costs:** To control token usage by the agent instead of submitting entire page contents as ‘evidence’, each fetched page is chunked, tokenized and ranked using BM25 and reranked using a cross encoder and only top-k results are fed to the agent.
- **Following Wikipedia API Usage Etiquettes:** The program follows Wiki API guidelines closely by implementing measures such as identifying itself properly, making sequential requests and applying exponential backoff.
- **Strictly an Information Retrieval Agent:** The Wiki Agent is responsible for information retrieval only – it is not involved in the generation of answers. The retrieved evidence is supplied to the answer generation agent.

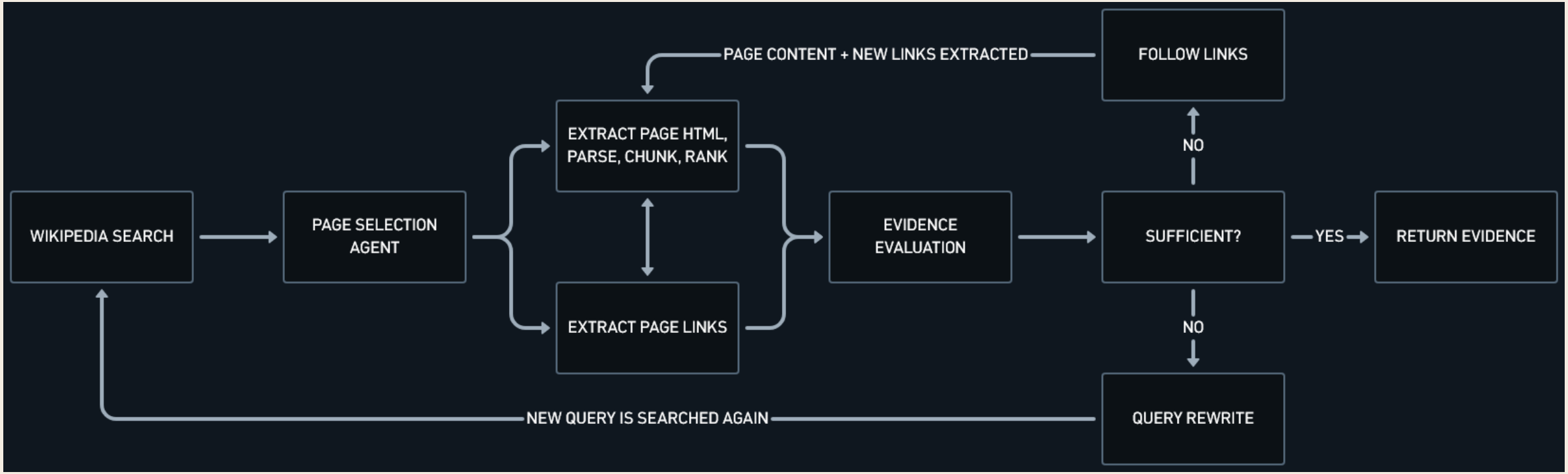
SYSTEM IMPLEMENTATION

WIKIPEDIA AGENT – HOW DOES IT WORK?

1. Initial Search: When the agent is invoked a Wiki search is performed on the supplied seed query. Wiki search results returns article title, page ID, description and excerpt.
2. Page Selection: The search results are submitted to the page selection subagent. It selects, based on the seed query, up to 5 pages and returns a list of selected page IDs
3. Page Retrieval: There are multiple steps involved in retrieval including a localised BM25+reranking pipeline.
    1. Selected page IDs are used to retrieve full HTML pages.
    2. The HTML is parsed using BeautifulSoup4 and irrelevant HTML elements and page sections are removed.
    3. Content is chunked and ranked using BM25. Then cross encoder model is used to rerank and filter top-k chunks of article data. These chunks are now used as evidence for the seed query
    4. Separately links are extracted and filtered from these pages and passed separately to the agent.
4. Evidence Evaluation: If the evidence is judged to be sufficient then the pipeline execution is terminated and said evidence is sent as final evidence. If evidence is judged as insufficient then:
    1. The agent may choose to rewrite the query in which case the query rewrite subagent is called. This initiates another Wiki Search and the loop is repeated.
    2. The agent may choose to follow the links supplied, in which case each page's HTML is fetched (same as above). However, the control layer maintains a "visited links" list using which it filters out the already visited links.

SYSTEM IMPLEMENTATION

WIKIPEDIA AGENT – HOW DOES IT WORK?



SYSTEM IMPLEMENTATION

STATIC RAG PIPELINE

- The Hybrid Retrieval pipeline is operated upon a static corpus.
- RAG Systems that operate on a static corpus present multiple benefits - such as supplementing the parametric knowledge of a LLM with additional knowledge or enabling us to work on proprietary or confidential data etc.
- This is the central component of our second pipeline. It follows a 2-step approach to complete its operation.
- Step 1: A detached data ingestion pipeline that is responsible for preprocessing and converting the static corpus into vector embeddings. It involves multiple steps which will be detailed in the following steps.
- Step 2: The runtime pipeline that is responsible for using the seed query embedding to compare, retrieve and rank relevant documents from the vector index. To optimize different stages of the retrieval, step 2 pipeline utilizes both reference focused and interaction focused modelling.
- In the following slides let us look at each step of the retrieval process.

SYSTEM IMPLEMENTATION

STATIC RAG PIPELINE: DATA INGESTION PIPELINE

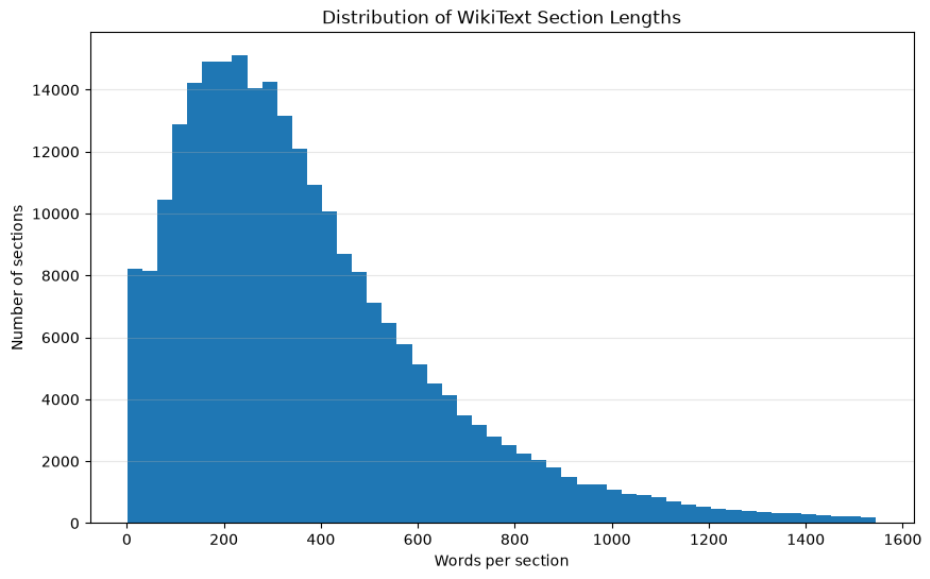
- Dataset: The dataset used for the project is WikiText acquired from Kaggle. It was split into training, testing and cross validation subsets. But since our focus is on retrieval instead of training, all three subsets were combined into a single corpus.
- Upon inspection it was discovered that all the articles were in a single file delimited using special sequence of characters:
  - Main Heading: = TOPIC NAME = - example: = Robert Boulter =
  - Sub-topic: = = SUBTOPIC = = - example: = = Career = =
  - Sub-heading: = = = SUBHEADING = = = - example: = = = 2000 – 2005 = = =
- Preprocessing Steps: First each article was extracted into a list, then leveraging the subsections' delimiters each subsection was split and stored in a dictionary that contained the subheading, subtopic and article title.
  - Done as a pre-emptive measure before chunking. This resulted in 256,982 individual sections.

```
{ "title": "Robert Boulter",
"sub_topic": "Career",
"sub_heading": "2000 – 2005",
"text": "Boulter began his career..."}
```

SYSTEM IMPLEMENTATION

STATIC RAG PIPELINE: DATA INGESTION PIPELINE

- **Chunking Strategy:** To decide the character/word length of each chunk, some basic metrics of individual word counts of each section was calculated and plotted. From this analysis, it was discovered that on average each section had about 400 words and 95% of the sections contained less than 1000 words.



- Therefore, it was decided that each chunk would contain at most 1000 words with no overlap.
- **The chunking method** would be recursive character-based splitting, since unlike fixed character size splitting, we can prevent chunks from being cut off mid sentence or mid word.
- Using the LangChain library, 270,296 chunks were created.

SYSTEM IMPLEMENTATION

STATIC RAG PIPELINE: DATA INGESTION PIPELINE

- Document Representation: Each chunk becomes an independent retrieval document. These chunks are stored in a pickle file as it is for future mapping to dense and sparse embeddings.
- Dense Embeddings: Chunks converted into dense vector embeddings of 768 dimensions using BGE-base-en-v1.5 model. These vectors are indexed using FAISS index.
  - Dense vectors can capture semantic relationships between a chunk and a query.
- Sparse Representation: Chunks are converted to sparse embeddings for BM25 relevance matching. Before indexing, each chunk was also tokenized to avoid classic NLP problems (e.g.: “Richard” and “Richard’s” would be considered different words without tokenizing.
  - Sparse vectors specialise in matching exact terms or keywords.

SYSTEM IMPLEMENTATION

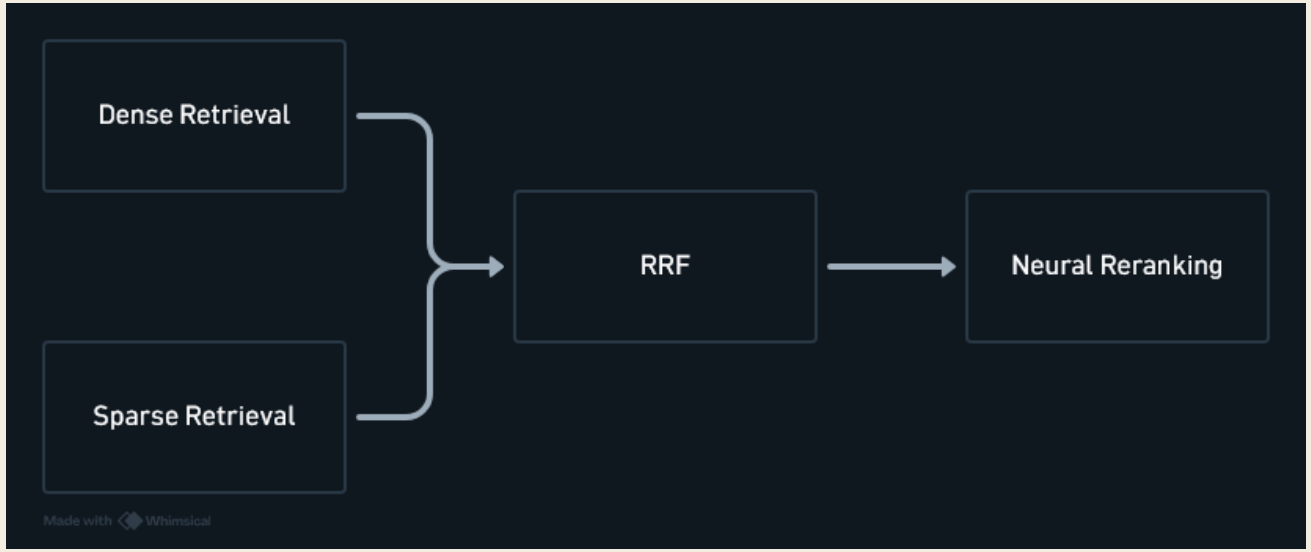
STATIC RAG PIPELINE: RETRIEVAL PIPELINE

- The retrieval pipeline implements a hybrid approach where both dense and sparse vector embeddings are retrieved based on the seed query and both sets of results are dynamically combined using reciprocal rank fusion.
- This phase of the pipeline utilizes representation focused modelling, aiming to achieve high recall@k (retrieve a higher volume of relevant documents at rank k) as well as interaction focused modelling to filter the retrieved documents and return only the most relevant documents trying to improve precision@k (ensure that a higher number of retrieved documents are relevant).
- Both dense and sparse retrievals are executed in parallel reducing end-to-end latency.
- Dense Retrieval:
  - As discussed before each chunk is converted into dense embeddings and stored using the FAISS index.
  - During execution the seed query is also converted into a vector embedding of the same dimensions (using the same model).
  - We then utilize FAISS index to search, retrieve and rank based on the similarity score with the query embedding.
- Sparse Retrieval: Sparse vectors are retrieved and ranked according to BM25 relevance score.
- Reciprocal Rank Fusion: After both phases of retrieval completes, there are two lists of rankings, one each from dense and sparse retrieval. To combine them we utilize Reciprocal Rank Fusion.
  - This step is necessary since both our lists of ranked documents could vary in either placement of each document (rank 1 in sparse but 3rd in dense) or some documents could be completely missing from either list.

SYSTEM IMPLEMENTATION

STATIC RAG PIPELINE: RETRIEVAL PIPELINE

- Neural Ranking: A relevant chunk can be semantically or lexically relevant, without being useful to answer the question.
  - Example: Query: How did the wars of the roses end? Retrieved document: Historians often disagree how the wars of the roses ended.
  - In this example, the document is semantically and lexically relevant but ultimately can contribute nothing to the answer.
  - Therefore, as the final step in the retrieval pipeline we employ neural ranking to rerank our retrieved chunks and return only the top-k most relevant chunks.
- For this step, a cross-encoder model, ms-marco-MiniLM-L-6-v2, is used.



SYSTEM IMPLEMENTATION

ANSWER GENERATION ENGINE

- The **primary responsibility** of the answer generation engine is to interface with the user, especially in the cases of generating the final answer to the user's query as well as requesting the user to provide further clarification for their prompt.
- Its **secondary responsibility** is to generate precise and brief answers for the intermediate objectives. In this stage it does not interact with the user.
- To fulfil these roles, the answer generation engine has been largely divided into three functions:
  - **Evidence grounded answer generation** – A set of evidential documents are submitted to the engine along with the user's query and it must generate an answer grounded by the evidence.
  - **Request user to provide clarification** – If invoked by the query handler agent, its task is to analyse the reason for clarification and request the user to provide clarification while providing an explanation.
  - **Intermediate answer generation –** Based on the retrieval plan generated by the planner, if necessary, the module will generate a brief and exact answer for the intermediate objective(s).
- **Prompting**: Each task has separate prompts containing their relevant instructions. Techniques such as few shot prompting has been implemented throughout.

EVALUATION

EVALUATION FRAMEWORK – LLM-AS-A-JUDGE

- Detached Evaluation framework inspired by the principles and metrics of RAGAS.
- Designed to evaluate 4 core qualities of the pipeline – Query Understanding, Planning, Evidence Retrieval and Answer Generation.
  - The score for each component was calculated after it finished execution which allowed to measure the scores of each component even if the pipeline eventually failed (for example in cases where evidence was insufficient).
- The system is built around the concept of LLM-as-a-judge, where we use an LLM to assess the quality of the work and convert it into numerical scores.
- Evaluation is conducted by comparing the system's answers and evidence to a precompiled dataset with ground truth answers and evidence. The dataset is discussed in the following slides.
- Design: The framework is divided into two distinct categories:
  - Benchmark System: A python module that extracts the first 100 questions from the dataset and automates the system, while also collecting information such as the query, the retrieved evidence, generated answer and end to end latency. Total token usage was attempted but not included in the final version. The collected information is stored in persistent storage.
  - Evaluation System: The benchmark data is compared against the ground truth data for each question and the LLM scores each answer and retrieved evidence according to the metrics.

EVALUATION

EVALUATION FRAMEWORK – EVALUATION METRICS

- Four distinct areas were focused on to judge each component of the project.
  - Query understanding was evaluated through intent, entity, concept, relationship, and completeness scores.
  - Planning was evaluated according to objective decomposition, dependency identification, seed-query generation, and execution.
  - Evidence was evaluated using context precision, context relevance, context recall, and evidence sufficiency.
  - Generated answers were evaluated for faithfulness, relevance, and correctness
- Evaluation Metrics:
  - Context Precision – How relevant the retrieved evidence is to the question
  - Context Recall – How much of the information needed to answer the question is present in the retrieved evidence.
  - Faithfulness – Whether the generated answer is supported by the retrieved evidence
  - Answer Relevancy – Whether the answer directly addresses the question
  - Answer Correctness – Whether the answer agree with the reference answer

EVALUATION

EVALUATION FRAMEWORK – BENCHMARK DATASET

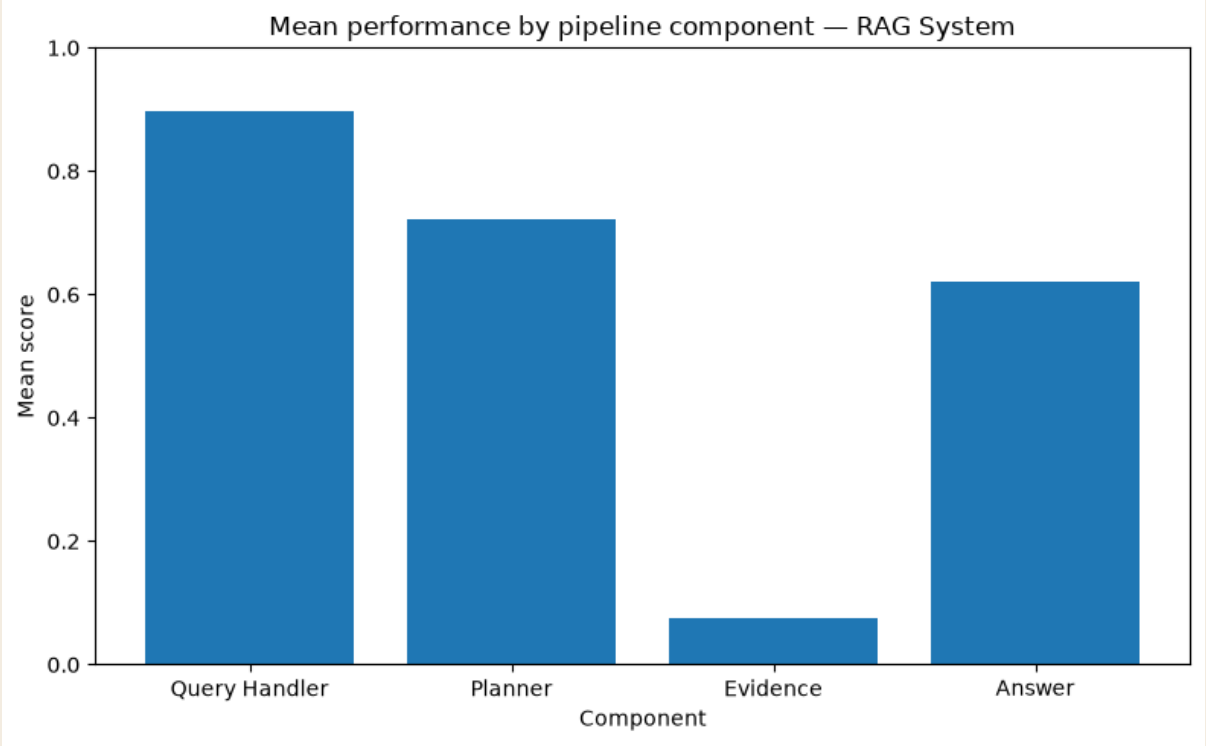
- For benchmark and evaluation, the dataset 2WikiMultiHopQA, a multi hop question-answer dataset was chosen.
- Sourced from HuggingFace library.
- The dataset contains questions that are based on information from different Wikipedia articles. Each datapoint contains a question, a ground truth answer and set of supporting evidence.
- The dataset contains 4 types of questions:
  - Bridge Comparison: Compares two entities where the required information can be obtained through an intermediate objective. Example: *Who was born before: the author of The Lord of the Rings or the drummer of the band The Beatles?*
  - Comparison: Compares the attributes/characteristics of two entities. Example: *Which is larger the population of Vienna or the number of people who attended the FIFA World Cup 2022 Finals?*
  - Compositional: Combines multiple facts to get to an answer. Example: *Where was the current president of Italy born?*
  - Inference: Question that requires the LLM to conclude a fact not directly present in the final sets of evidence. Example: *What are the parallels between the TV Show Game of Thrones and the historical conflict Wars of the Roses?*

EVALUATION

RESULTS – STATIC RAG

Average Performance by component:

Component	Score
Query Handler	0.898
Planner	0.722
Evidence Retrieval	0.075
Answer Generation	0.620



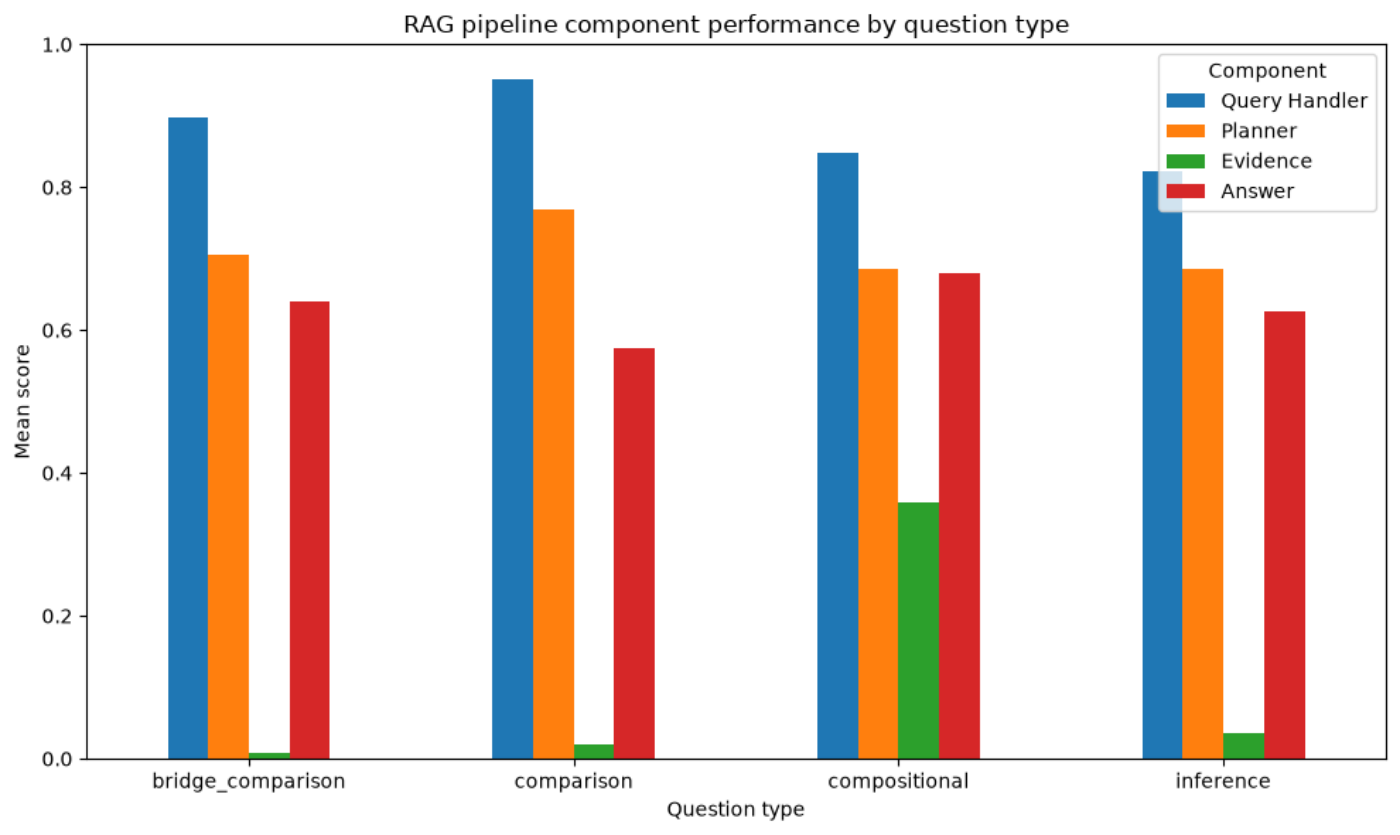
- Strong performance in query handling and planning.
- Exponentially weaker in evidence retrieval
- Answer generation performed above average
- CONCLUSION: When the system had enough evidence, the final answer remained grounded in evidence.

EVALUATION

RESULTS – STATIC RAG

Individual Scores:

Metric	Score
Context Precision	0.104
Context Recall	0.095
Context Relevance	0.051
Evidence Sufficiency	0.049
Answer Faithfulness	0.820
Answer Correctness	0.331



Looking at individual scores:

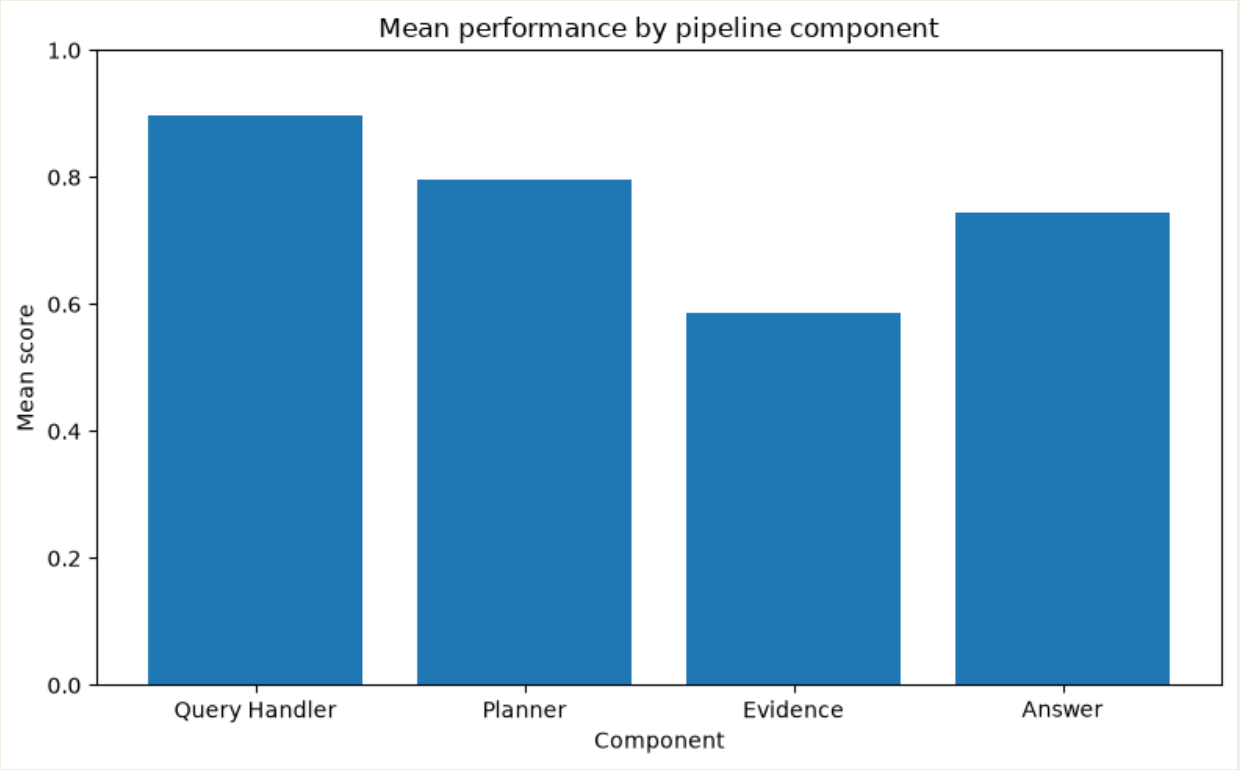
- Evidence gathering was of low quality or missing
- Due to insufficient evidence, answer quality suffered
- However, answers stayed largely grounded whenever evidence was presented.

EVALUATION

RESULTS – WIKIPEDIA AGENT

Average Performance by component:

Component	Score
Query Handler	0.897
Planner	0.796
Evidence Retrieval	0.586
Answer Generation	0.744



- Strong performances by all three components.
- Evidence retrieval is substantially better but still lacking.
- **CONCLUSION**: When the system had enough evidence, the overall system performance increased dramatically.

EVALUATION

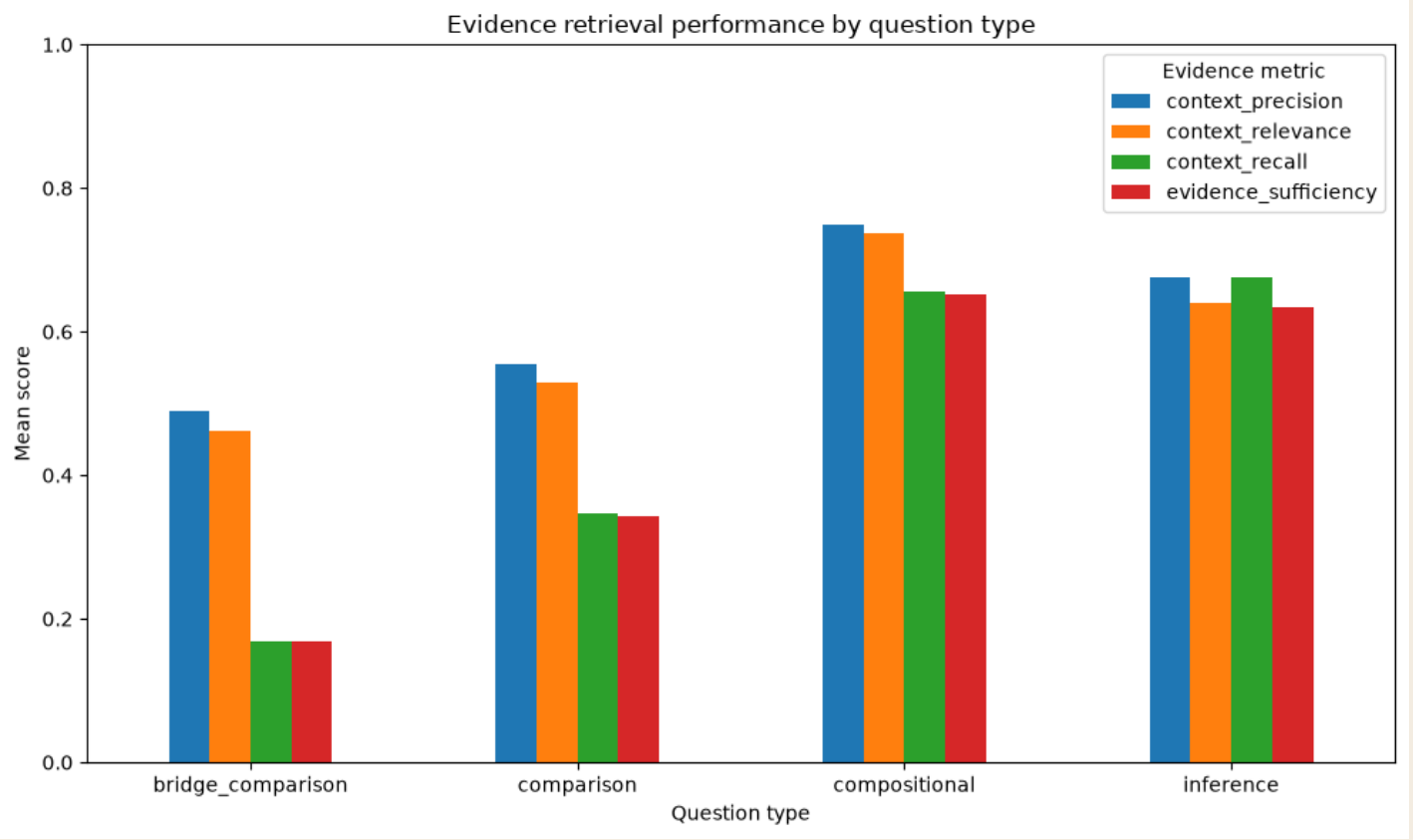
RESULTS – WIKIPEDIA AGENT

Individual Scores:

Metric	Score
Context Precision	0.661
Context Recall	0.640
Context Relevance	0.526
Evidence Sufficiency	0.518
Answer Faithfulness	0.858
Answer Correctness	0.554

Looking at individual scores:

- Evidence gathering has improved
- Answer correctness is proportional to evidence metrics.
- Moreover, answers stayed largely grounded whenever evidence was presented.



CONCLUSION: Evidence gathering has improved due to which answer quality has also improved. System stays highly grounded to evidence while answering. Bridge and regular comparison questions point out a pattern of flaw worth exploring.

EVALUATION

RESULTS – A COMPARISON OF TWO APPROACHES

- Looking at the component wise scores side by side for each pipeline, we can immediately notice the stark difference in evidence processing.
- Consequently, another point of observation is that when evidence gathering is better answer quality also increases.
- We may conclude that the performance of the system produces good quality answers if the sufficient evidence is present.

Component	Wiki Agent	Static RAG
Query Handler	0.897	0.898
Planner	0.796	0.722
Evidence Retrieval	**0.586**	**0.075**
Answer Generation	0.744	0.620

- When looking at system latency, the static RAG system has a clear advantage.
- The outliers are mostly caused by models getting stuck in thinking loops or any of the implemented limits.

End-to-End Latency (s)	Wiki Agent	Static RAG
Minimum	41.15	29.59
Maximum	3952.45	1052.44
Average	263.40	102.62
Median	129.14	45.74

CONCLUSIONS

- Static RAG's poor evidence retrieval: The primary reason, after inspecting the execution logs, seems to be that for most questions, there was no data available in the static corpus. Although for our purposes we could change our benchmark dataset or add the data missing data to the static corpus and measure again.
    - However, through this outcome we can prove one of the fundamental flaws of a static corpus RAG – the availability and maintenance of data will always determine how the system works. The more the data grows the more the chances that the system fails due to missteps in maintenance.
- Wikipedia Agent's point of failure: Although the wiki agent can course correct itself in case it starts wandering, its single point of failure lies with the Wikipedia search function. If for a topic the Wikipedia search itself is not up to standard, then the agent can never retrieve quality evidence. For example, consider the question "Who was the mother of the director of the movie 'Polish-Russian War'"? In this case the agent wanders and eventually lands on the wrong set of evidence. But upon inspection it was discovered that, perhaps due the ambiguity of the question, even Wikipedia Search does not return the correct search result.
- Wiki Agent's Latency: Due to numerous limitations placed upon the wiki agent, mainly due to following the Wikipedia API's etiquette guidelines, the Wiki agent suffers from substantial delays. This remains the system's greatest shortcoming.
- Other components: All the other components – query handler, planner and answer generation engine are functioning very well. Implementing prompting techniques such as few shot prompting could be the reason.



THANK YOU